SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Programme: **B.E / B.Tech**

Course Code: **CSA17**

Course Name: **ARTIFICIAL INTELLIGENCE**

Submission Date: 16-11-2026

Date of Issue : 5-11-2026 Max Marks: **50 Marks**

ASSIGNMENT 3

Course Outcomes

On successful completion of the course, students will be able to:

CO	Course Outcome Statement
CO3	Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making.

Evidence Rule Common to All Assignments

Every implemented task must be supported by: **Source Code → Test Input → Expected Behaviour → Actual Output → Performance Metrics (table/graph over at least three input sizes or seeds) → Interpretation.** A screenshot showing only a final answer is not sufficient. Operation counts and task metrics are primary evidence; execution time is secondary. Wherever a task says *Predict*, the prediction must be written before the experiment is run. Programming language: Python 3 (Jupyter / Colab).

ASSIGNMENT 3 – CO3

ARTIFICIAL INTELLIGENCE – CSA17

CO3 Coverage: Logical agents – Propositional logic – Reasoning patterns in propositional logic – Resolution – First-order logic – Knowledge engineering in FOL – Inference in first-order logic – Propositional versus first-order inference – Unification and lifting – Forward chaining – Backward chaining – Resolution.

Mode: Pair Work – 2 Students per Group
Duration: 1 Week
Total Marks: 50

Submission Date: _____

Pair-Work Principle

Both students are responsible for understanding the complete assignment. Each task contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must **not** divide the assignment task-wise. Both students must participate in every task. The individual responsibilities alternate across tasks so that both students show evidence of: **Design → Implementation → Analysis → Evaluation**.

TASK 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns (6 Marks)

Part A – Knowledge Base and Model Checking in the Wumpus World

Shared Group Work: Together encode the rules of an $n \times n$ Wumpus world in propositional logic (pit, wumpus, breeze and stench symbols for every square and the rules relating percepts to neighbouring hazards). Implement a truth-table entailment checker and query it with percept sequences that you design, for example, "Is square [2,2] safe?".

Student A – Design and Mathematical Analysis

- Count the proposition symbols and sentences as a function of n
- Derive the number of models 2^m and identify the minimal set of symbols relevant to one query
- Explain what soundness and completeness mean for model checking

Student B – Experimental Validation

- Instrument the checker to record models enumerated, KB evaluations and time
- Run it for n = 2, 3, 4 (state where the full symbol set becomes infeasible) and again with relevant-symbol restriction
- Include one entailed query, one query whose negation is entailed and one query for which neither is entailed

Part B – Reasoning Patterns

Shared Group Work: Using the same Wumpus-world KB, derive the safety of at least two squares by applying inference rules (Modus Ponens, And-Elimination, Unit Resolution) instead of model checking.

Student A – Experimental Investigation: Implement the inference-rule derivation, record the number of inference steps and compare with the model-checking cost for the same query.

Student B – Soundness Reasoning: Prove the soundness of each rule used with a truth table and explain why a derived sentence must hold in every model of the KB, and why "not derived" is not the same as "false".

Cross-Understanding Requirement: Each student must be able to explain the other student's component.

Evidence Expected: KB listing, symbol-count derivation, checker code and outputs for the three query types, inference-rule derivations with step counts, soundness tables and a comparison of the two reasoning approaches.

TASK 2 – The Quality of Reasoning Method: Model Checking versus Resolution (7 Marks)

Shared Group Work: Implement both a truth-table model checker and a propositional resolution prover (CNF conversion followed by resolution refutation). Design a smart-building access-control policy of at least 10 rules (badge validity, time of day, visitor status, emergency override) that contains implication, biconditional and disjunction.

The group must design test knowledge bases that make each method look strong or weak. Do not use a single sample KB.

Student A – Resolution Investigation

- Convert three sentences to CNF by hand and construct a resolution proof for one entailed query

- Record clauses generated, resolvents attempted, tautologies discarded, subsumed clauses removed and time as the KB grows (implication chains of length 10, 20, 40, 80)
- Test a query that is not entailed and explain why the process terminates

Student B – Model-Checking Comparative Analysis

- Record models enumerated and time for the same KBs and queries
- Identify a KB family on which model checking is faster than resolution and a family on which it is far slower, and explain what structural property causes the difference

Joint Design Question

If you were designing a reasoning component whose KB size and query types are not known in advance, what information would you want before deciding between these two methods? Support your answer with your own measurements.

Evidence Expected: Manual CNF conversion, proof tree, both implementations, clause and model counts across KB sizes, designed KB families and the joint design discussion.

TASK 3 – Breaking the Expressiveness Barrier: Knowledge Engineering in First-Order Logic (7 Marks)

Problem Context – University Exam-Eligibility Advisor

Regulations decide whether a student may write an examination (minimum attendance, prerequisites cleared, fees paid, disciplinary status, approved leave).

Shared Group Work: Follow the knowledge-engineering process (identify the task, assemble knowledge, decide vocabulary, encode general knowledge, encode the instance, pose queries, debug the KB). Build two encodings of the same regulations: a **propositional** one (one set of rules per student and course) and a **first-order** one (general axioms plus facts).

Student A – Design Derivation

- Define the FOL vocabulary and write at least 12 axioms with quantifiers, including one existential statement
- Derive how the number of sentences in the propositional encoding grows with students *s* and courses *c*, and show how the FOL axioms remain constant

Student B – Crossover Investigation

- Design at least 15 competency queries (5 that succeed, 5 that fail, 5 returning bindings) and document at least three KB defects found and fixed
- Measure KB size and inference time for both encodings as *s* and *c* grow, and locate the range of sizes at which the FOL encoding becomes preferable despite unification overhead

Joint Conclusion

Why does greater expressiveness not automatically give better reasoning performance for every KB size? Support your answer with your measurements.

Evidence Expected: Vocabulary and axioms, growth derivation, query suite with expected and actual results, defect log, size/time tables and a reasoned conclusion.

TASK 4 – The Reality of Unification and Lifted Inference (7 Marks)

The responsibilities in this task are reversed compared with Task 3.

Shared Group Work: Implement UNIFY (with substitution composition and an optional occurs check), lifted Modus Ponens, and propositionalisation of a small FOL KB (instantiating universal sentences with all ground terms).

Student B – Logical Design and Correctness

- Derive the most general unifier by hand for five pairs of terms that you design, including a pair where the occurs check matters and one where standardising apart is essential
- Explain why lifted inference is sound and why it can avoid generating ground instances

Student A – Experimental and Engineering Analysis

- Measure the number of ground clauses as the number of constants grows (k = 3 to 30) and as predicate arity grows
- Introduce a function symbol and show how the number of ground terms grows with the term-depth bound
- Compare inference steps and time for lifted versus propositional inference on the same query

Joint Engineering Decision

Under what conditions would you propositionalise a knowledge base despite the availability of lifted inference? State the conditions using your own evidence.

Evidence Expected: Unifier code and tests, hand derivations, ground-clause growth tables and plots, a lifted-versus-propositional comparison and the engineering decision.

TASK 5 – Upgrading Resolution from Propositional to First-Order Logic (6 Marks)

In Task 2, propositional resolution was applied to a policy KB. Review that solution before attempting this task.

Shared Group Work: Using the following English statements, design a first-order resolution prover:

1. Every employee who handled a restricted document is authorised.
2. Every document containing patient data is restricted.
3. Some employee handled every document that was audited.
4. Report R1 contains patient data and R1 was audited.

Convert the statements to FOL and CNF (with Skolemisation), implement resolution using your Task 4 unifier, and prove that an authorised employee exists.

Student A – Conversion and Proof Design: Show every conversion step, justify the Skolem function or constant, draw the proof tree and implement one resolution strategy of your choice (unit preference, set-of-support or input resolution).

Student B – Comparative Evidence: Record clauses generated, resolution steps, proof length and unifications with and without the strategy. Pose a second query that is **not** entailed and report the behaviour of your prover under a step cap. Compare with the propositional prover of Task 2 on an equivalent small domain.

Cross-Understanding Requirement

- Student B must explain why Skolemisation preserves satisfiability but not logical equivalence
- Student A must explain what the experimental evidence shows about the cost of lifting resolution to first-order logic, and what the non-entailed query shows about the decidability of first-order entailment

Evidence Expected: Conversion steps, proof tree, prover code, strategy comparison, behaviour on the non-entailed query and responses to the cross-understanding questions.

TASK 6 – Advanced Inference Mini-Investigations (7 Marks)

Both students investigate a different inference strategy on the same knowledge base (an organisation hierarchy with reports-to relations and approval rules, about 10 definite-clause rules, with 100, 1,000 and 10,000 facts).

Student A – Forward Chaining Investigation

- Implement forward chaining to a fixed point
- Record facts inferred, rule firings, redundant derivations, iterations to fixed point and the percentage of derived facts irrelevant to the query
- Explain what happens to the work when the query concerns only a very small part of the KB

Student B – Backward Chaining Investigation

- Implement backward chaining with substitutions
- Engineer a left-recursive rule on which naive backward chaining never terminates, implement a remedy (loop check, depth bound or memoisation) and evaluate its cost
- Record goals expanded, unifications, maximum depth and repeated subgoals

Verification Questions

- Student B must explain: *Why is the choice of inference direction central to performance on this KB?*
- Student A must explain: *Why can backward chaining remain efficient even when the KB contains many facts irrelevant to the query?*

Evidence Expected: Two working engines, metric tables for the three fact sizes, the non-termination demonstration, the remedy, and correct responses to the verification questions.

DAY-4 DESIGN CHALLENGE

TASK 7 – Design a Knowledge-Based Reasoning Component from First Principles (10 Marks)

Problem Context – Hospital Medication-Safety Advisor

Given a patient record (age, allergies, current medicines, conditions) and a newly prescribed drug, the agent must decide whether the prescription is safe and explain why. The initial knowledge base has about 40 rules. The hospital plans to grow it to 2,000 rules and 50,000 patients.

This problem must be solved collaboratively. Neither student may work independently and merge the answers later.

Stage 1 – Independent Design Proposal – 1 Mark

Before discussing with each other, Student A and Student B must each independently propose: the representation (propositional or FOL), vocabulary, inference method (forward chaining, backward chaining or resolution) and how explanations will be produced. Each initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect.

Stage 2 – Compare the Two Designs – 1 Mark

Identify where the designs agree and differ, their strengths and weaknesses, and which ideas will be retained.

Stage 3 – Joint Final Design – 3 Marks

Produce the final design: representation, vocabulary, inference method, indexing or agenda strategy for scale, explanation mechanism and complete pseudocode.

Stage 4 – Divide the Validation Responsibilities – 3 Marks

- **Student A – Theoretical Validation:** soundness and completeness of the chosen inference, complexity of a query as a function of KB size, and the effect of restricting the KB to definite clauses.

- **Student B – Experimental Validation:** at least 30 test cases including dangerous combinations, KB sizes of 40, 400 and 2,000 rules, metrics (query latency, inference steps, memory, explanation length) and a baseline comparison, with code and outputs as proof.

Stage 5 – Cross-Defence – 1 Mark

Student A explains the experimental evidence supporting the group's design. Student B explains why the theoretical complexity result applies to the group's chosen inference.

Stage 6 – Joint Evaluation – 1 Mark

Why is the problem suitable for a logical agent? What are the limits of the representation (uncertainty, incomplete data)? What should an AI designer check before choosing a knowledge-based approach?

CONTRIBUTION RECORD

Each pair must include this table at the end of the submission, reflecting the actual work completed.

Task	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Task 1			
Task 2			
Task 3			
Task 4			
Task 5			
Task 6			
Task 7			

MARK DISTRIBUTION – ASSIGNMENT 3

Task	Assessed Area	Marks
Task 1	Logical agents, entailment and reasoning patterns	6
Task 2	Model checking versus resolution	7
Task 3	Knowledge engineering in FOL	7
Task 4	Unification and lifted inference	7
Task 5	First-order resolution	6
Task 6	Forward and backward chaining	7
Task 7	Day-4 Design Challenge	10
	TOTAL	**50**

Submission Requirements: one report per pair for Tasks 1–7 with the contribution record, source code for all implementations, raw test data, and the tables/graphs used. Operation counts (models, clauses, inference steps) are primary evidence; time is secondary.

INDIVIDUAL VERIFICATION AND VIVA MODEL

Primary contribution does not imply exclusive responsibility. A student must not answer “My partner did this section.” For each student conduct a brief 4–5 minute verification:

1. **Primary responsibility** – one technical question from the component the student led.
2. **Partner’s responsibility** – one question from a component primarily completed by the partner.
3. **Transfer question** – modify a condition and ask for the consequence, for example: “If you remove the occurs check, which of your test pairs now gives a wrong answer?” or “If the KB grows to 100,000 facts and queries are rare, would you change from forward to backward chaining?”

Overall Assessment Expectation: Represent → Infer → Implement → Measure → Debug → Evaluate → Defend.

ARTIFICIAL INTELLIGENCE – ASSIGNMENT 3 (CO3)

Answer Section

Name: Sahana S **Register No:** 192321131

Course Code: CSA17 **Course:** Artificial Intelligence

Submission date: 16-11-2026

Document arrangement: The seven pages of the supplied assignment are retained unchanged. This answer section is appended after the original question paper.

Implementation note: The code examples are Python 3 reference implementations. Performance tables below are analytical/sample measurements for the stated finite test setups; run the included code in Jupyter/Colab and replace sample timings with the actual runtime values from your system before submission.

TASK 1 – LOGICAL AGENTS: KNOWLEDGE BASES, ENTAILMENT AND REASONING PATTERNS

Part A – Knowledge base and model checking

Vocabulary. For every square (i,j), use Pij = pit, Wij = Wumpus, Bij = breeze and Sij = stench. A square is safe when neither a pit nor a Wumpus is present: Safeij ≡ ¬Pij ∧ ¬Wij.

Wumpus rules. A breeze is present exactly when at least one orthogonally adjacent square contains a pit. A stench is present exactly when at least one orthogonally adjacent square contains the Wumpus. For each square x, let Adj(x) be its in-bounds neighbours.

```
B_x <-> OR(P_y for y in Adj(x))
S_x <-> OR(W_y for y in Adj(x))
Safe_x <-> (not P_x and not W_x)
```

For an n×n grid there are $4n^2$ percept/hazard symbols. If safety atoms are explicitly included, there are $5n^2$ symbols. With the biconditional expanded into implications, each breeze and stench definition contributes two sentences, and each safety definition contributes one sentence: $5n^2$ sentences, excluding initial-world constraints and observed percept facts. If only P, W, B and S are primitive, $m = 4n^2$ and the unconstrained truth-table space contains 2^{4n^2} models. With explicit Safe atoms, $m = 5n^2$ and the space is 2^{5n^2}.

Minimal relevant symbols. For a query about square (2,2), retain P22 and W22 plus the hazard symbols in squares connected to the observed breeze/stench facts through the relevant adjacency rules. Symbols unrelated to the query and evidence can be omitted only when they cannot affect the query's truth.

Soundness and completeness. Model checking is sound when every sentence entailed by the checker is true in all models satisfying the KB. It is complete when every sentence true in all KB models is reported as entailed. Exhaustive truth-table checking over the complete vocabulary has both properties.

Reference truth-table checker

```
from itertools import product
def entails(symbols, kb, query):
    models = 0
    kb_models = 0
    query_true = True
    query_false = True
    for bits in product([False, True], repeat=len(symbols)):
        m = dict(zip(symbols, bits)); models += 1
        if all(s(m) for s in kb):
            kb_models += 1
            query_true &= query(m)
            query_false &= (not query(m))
    if kb_models == 0: return "INCONSISTENT KB", models, kb_models
    if query_true: result = "ENTAILED"
    elif query_false: result = "NEGATION ENTAILED"
    else: result = "NEITHER ENTAILED"
    return result, models, kb_models
```

For an illustrative 2×2 test, let the evidence imply ¬P11, ¬W11, ¬P12 and ¬W12. Query Safe11 is entailed. Query P11 is refuted (its negation is entailed). Query P22 is neither entailed nor refuted if no rule or percept constrains square (2,2).

Part B – Inference patterns and soundness

Derivation 1. From ¬P11 and ¬W11, And-Elimination gives each conjunct separately; together they establish Safe11 = ¬P11 ∧ ¬W11.

Derivation 2. If B11 is false and the breeze rule is B11 ↔ (P12 ∨ P21), then ¬B11 and the rule yield ¬P12 and ¬P21 by unit resolution. The same pattern applies to stench and Wumpus locations.

Modus Ponens: P→Q, P ■ Q. Truth table: the only row where both premises are true has Q=true. **And-Elimination:** P∧Q ■ P and P∧Q ■ Q; whenever the premise is true, both conjuncts are true. **Unit Resolution:** (L∨A), ¬L ■ A; if ¬L is true, L is false, so A must be true for the first clause to hold.

A derived sentence must hold in every model of the KB because each inference rule preserves truth in all models. “Not derived” does not mean “false”: the inference procedure may be incomplete, may have a restricted rule set, or may simply lack enough facts. The sentence may be true in some models and false in others.

Comparison and expected instrumentation

Record (i) all assignments enumerated, (ii) assignments satisfying the KB, (iii) KB sentence evaluations, (iv) inference-rule firings and (v) elapsed time. Full enumeration grows exponentially: for the $4n^2$ vocabulary, n=2 gives 65,536 assignments, n=3 gives 68,719,476,736 and n=4 gives 18,446,744,073,709,551,616. Thus n=3 is already impractical for naive exhaustive enumeration; relevant-symbol restriction is essential.

For the small derivations above, inference requires only a few rule applications, whereas model checking explores every assignment in the selected vocabulary. The exact count depends on the selected KB and must be obtained by running the checker.

TASK 2 – MODEL CHECKING VERSUS RESOLUTION

Smart-building access-control knowledge base

Let V = valid badge, D = daytime, E = employee, G = guest, R = registered visitor, H = host approval, M = security monitoring active, X = emergency override, T = restricted area, A = access granted, and L = access logged. Example policy rules:

• 1. $V \wedge D \wedge E \rightarrow A$ (valid employee access during daytime).

• 2. $V \wedge H \wedge G \rightarrow A$ (approved guest access).

• 3. $G \rightarrow R$ (guests must be registered).

• 4. $R \wedge H \rightarrow A$ (registered visitor with host approval).

• 5. $X \rightarrow A$ (emergency override grants access).

• 6. $A \rightarrow L$ (every granted entry is logged).

• 7. $T \wedge \neg M \rightarrow \neg A$ (restricted area is denied when monitoring is off).

• 8. $A \rightarrow V \vee X$ (ordinary or emergency authorization is required).

• 9. $A \leftrightarrow (V \wedge D \wedge E) \vee (V \wedge H \wedge G) \vee X$ (policy-level access definition for the simplified test case).

• 10. $\neg D \wedge \neg X \rightarrow \neg A$ (outside daytime, no access without override).

The biconditional in rule 9 is a simplified policy definition; when combining it with other rules, ensure the test KB does not accidentally create contradictory policies.

CNF conversion examples

1) $(V \wedge D \wedge E) \rightarrow A \equiv \neg(V \wedge D \wedge E) \vee A \equiv \neg V \vee \neg D \vee \neg E \vee A$.

2) $X \rightarrow A \equiv \neg X \vee A$.

3) $A \leftrightarrow (V \wedge H) \equiv (\neg A \vee V) \wedge (\neg A \vee H) \wedge (A \vee \neg V \vee \neg H)$.

Resolution refutation

To prove A from X and $(\neg X \vee A)$, add $\neg A$. Resolve $(\neg X \vee A)$ with $\neg A$ to obtain $\neg X$; resolve $\neg X$ with X to obtain the empty clause ■. Therefore KB ■ A.

Propositional resolution reference code

```
def resolve(c1, c2):
    out = set()
    for lit in c1:
        comp = lit[1:] if lit.startswith("~") else "~"+lit
        if comp in c2:
            r = (set(c1)-{lit}) | (set(c2)-{comp})
            if any((x[1:] if x.startswith("~") else "~"+x) in r for x in r):
                continue
            out.add(frozenset(r))
    return out

def resolution_entails(clauses, query, step_cap=10000):
    clauses = set(map(frozenset, clauses))
    clauses.add(frozenset({"~"+query}))
    steps = 0
    while steps < step_cap:
        new = set()
        items = list(clauses)
        for i in range(len(items)):
            for j in range(i+1, len(items)):
                steps += 1
                for r in resolve(items[i], items[j]):
                    if not r: return True, steps, len(clauses)
                    new.add(r)
                if steps >= step_cap: break
            if steps >= step_cap: break
        if new.issubset(clauses) or not new:
            return False, steps, len(clauses)
        clauses |= new
    return None, steps, len(clauses)  # cap reached
```

A query not entailed does not produce the empty clause. For a finite propositional clause set, resolution reaches saturation (no new clauses) or derives ■; saturation without ■ means the query is not entailed.

Comparison and design decision

Model checking enumerates 2^m assignments for m proposition symbols. It can be faster for a tiny vocabulary or when a satisfying countermodel is found early. Resolution can be much faster when the KB is sparse and a short proof exists, because it manipulates clauses rather than all assignments. Resolution can become expensive when many clauses interact and generate many resolvents.

Before choosing, measure vocabulary size, clause count, clause width, implication-chain length, expected query frequency, whether queries are entailment or satisfiability, memory limits, and whether countermodels or proofs are needed. Benchmark several KB families, not one example. A chain of implications often favours resolution; a tiny KB with few atoms can favour model checking.

TASK 3 – KNOWLEDGE ENGINEERING IN FIRST-ORDER LOGIC

University exam-eligibility advisor

Vocabulary: Student(x), Course(c), Attendance(x,c,a), MinimumAttendance(c,m), Prerequisite(c,p), Cleared(x,p), FeesPaid(x,c), DisciplinaryBlock(x), ApprovedLeave(x,c), Eligible(x,c), and Enrolled(x,c). Variables x,y denote students; c,p denote courses; a,m denote attendance values.

Twelve FOL axioms

1. $\forall x \forall c \forall a \forall m$ [(Student(x) $\wedge$ Course(c) $\wedge$ Attendance(x,c,a) $\wedge$ MinimumAttendance(c,m) $\wedge$ a$\geq$m $\wedge$ FeesPaid(x,c) $\wedge$ $\neg$DisciplinaryBlock(x) $\wedge$ PrerequisitesCleared(x,c)) $\rightarrow$ Eligible(x,c)].

2. $\forall x \forall c$ [Eligible(x,c) $\rightarrow$ Student(x) $\wedge$ Course(c)].

3. $\forall x \forall c$ [ApprovedLeave(x,c) $\rightarrow$ Student(x) $\wedge$ Course(c)].

4. $\forall x \forall c$ [FeesPaid(x,c) $\rightarrow$ Student(x) $\wedge$ Course(c)].

5. $\forall x \forall p$ [Cleared(x,p) $\rightarrow$ Student(x) $\wedge$ Course(p)].

6. $\forall c \forall p$ [Prerequisite(c,p) $\rightarrow$ Course(c) $\wedge$ Course(p)].

7. $\forall x \forall c$ [PrerequisitesCleared(x,c) $\leftrightarrow$ $\forall$p(Prerequisite(c,p) $\rightarrow$ Cleared(x,p))].

8. $\forall x \forall c$ [DisciplinaryBlock(x) $\rightarrow$ $\neg$Eligible(x,c)].

9. $\forall x \forall c$ [$\neg$FeesPaid(x,c) $\rightarrow$ $\neg$Eligible(x,c)].

10. $\forall x \forall c \forall a \forall m$ [(Attendance(x,c,a) $\wedge$ MinimumAttendance(c,m) $\wedge$ a<m $\wedge$ $\neg$ApprovedLeave(x,c) $\rightarrow$ $\neg$Eligible(x,c)].

11. $\forall x \forall c$ [Enrolled(x,c) $\rightarrow$ Student(x) $\wedge$ Course(c)].

12. $\exists x \exists c$ [Student(x) $\wedge$ Course(c) $\wedge$ Enrolled(x,c)].

The attendance rule can be extended so approved leave modifies the attendance threshold according to institutional policy. This example treats approved leave as an exception to the low-attendance rejection.

Propositional versus FOL growth

A propositional encoding creates student-course-specific atoms and rules. If there are r rules per student-course pair, its rule count is approximately rsc, plus facts proportional to sc. FOL stores the general rules once (12 axioms here) and adds instance facts, approximately O(sc) in the worst case. The general axiom count stays constant as s and c grow.

Competency query suite

Successful (5): Is student S1 eligible for C1? Is S2's fee paid? Has S1 cleared prerequisite P1? Is C1 a course? Does S1 satisfy all prerequisites for C1?

Failing (5): Is S3 eligible with attendance below minimum and no leave? Is S4 eligible with unpaid fees? Is S5 eligible with a disciplinary block? Is a non-student eligible? Is a student eligible for an unknown course?

Binding (5): Which students are eligible for C1? Which courses can S1 write? Which students have paid fees? Which students cleared P1? Which student-course pairs satisfy attendance?

Illustrative defects and fixes: (1) Missing Course(c) facts caused valid queries to fail – add course instance facts. (2) A universal prerequisite rule accidentally required a student to clear every course – restrict the quantifier to courses that are prerequisites of the queried course. (3) Approved leave was ignored in the low-attendance rule – add $\neg$ApprovedLeave(x,c) to that rule.

Performance interpretation

For s students and c courses, compare the total number of stored sentences and inference time at (10,5), (100,20), and (500,50). Propositional rules are duplicated per pair; FOL reuses general axioms but pays for unification and substitution. FOL becomes preferable when repeated instances share the same general rules and the cost of duplicated propositional rules exceeds unification overhead. There is no universal crossover point; determine it empirically on the chosen implementation.

Greater expressiveness does not guarantee faster reasoning. FOL compactly represents general patterns but introduces variable matching, substitutions, and potentially larger search spaces. For tiny finite domains, propositionalisation may be simpler and faster.

TASK 4 – UNIFICATION AND LIFTED INFERENCE

Five most general unifier examples

• 1. Knows(John,x) and Knows(John,Mary): MGU {x/Mary}.

• 2. Teaches(x,AI) and Teaches(Anu,y): MGU {x/Anu, y/AI}.

• 3. Parent(x, f(y)) and Parent(Asha, f(Ravi)): MGU {x/Asha, y/Ravi}.

• 4. P(x,x) and P(Lee,Mina): failure, because x would need to be both Lee and Mina.

• 5. Loves(x) and Loves(f(x)): failure under occurs check, because x would have to equal f(x).

Standardising apart is essential when combining clauses that reuse variable names. For example, Parent(x,y) and Parent(y,z) must first be renamed to Parent(x1,y1) and Parent(y2,z2); otherwise the two clauses may accidentally share variables that were intended to be independent.

Unification algorithm

```
def walk(x, subst):
    while isinstance(x, str) and x in subst:
        x = subst[x]
    return x

def occurs(v, term, subst):
    term = walk(term, subst)
    if v == term: return True
    if isinstance(term, tuple):
        return any(occurs(v, t, subst) for t in term[1:])
    return False

def unify(x, y, subst=None):
    subst = {} if subst is None else dict(subst)
    x, y = walk(x, subst), walk(y, subst)
    if x == y: return subst
    if isinstance(x, str) and x[:1].islower():
        if occurs(x, y, subst): return None
        subst[x] = y; return subst
    if isinstance(y, str) and y[:1].islower():
        if occurs(y, x, subst): return None
        subst[y] = x; return subst
    if isinstance(x, tuple) and isinstance(y, tuple) and len(x)==len(y) and x[0]==y[0]:
        for a,b in zip(x[1:], y[1:]):
            subst = unify(a,b,subst)
            if subst is None: return None
        return subst
    return None
```

This compact representation uses tuples such as ('Parent','Asha','Ravi'). Lowercase strings represent variables; constants begin with uppercase letters. A production implementation should use explicit Variable, Constant and Function data types.

Lifted Modus Ponens

Given $\forall x\ (P(x) \rightarrow Q(x))$ and P(Asha), unify P(x) with P(Asha), obtaining {x/Asha}; apply the substitution to Q(x) and infer Q(Asha). This is sound because the substitution makes the antecedent identical to the known fact. Lifted inference avoids creating every possible ground instance before it is needed.

Grounding growth

For a predicate of arity r and k constants, there are k^r possible ground atoms. With k=3,10,20,30 and arity 2, the counts are 9,100,400,900; for arity 3 they are 27,1,000,8,000,27,000. With a function symbol f and one constant a, terms of depth at most d include a, f(a), …, f^d(a): d+1 terms. With branching function symbols, term growth is faster.

Lifted inference is preferred when the domain is large and only a small number of substitutions are relevant. Propositionalisation is reasonable when the domain is small and finite, all constants are known, exhaustive checking is required, or a propositional solver is substantially better optimised. Compare inference steps, generated ground clauses and time on the same queries.

TASK 5 – FIRST-ORDER RESOLUTION

FOL translation

• 1. ∀x∀d [(Employee(x) ∧ Handled(x,d) ∧ Restricted(d)) → Authorised(x)].

• 2. ∀d [(ContainsPatientData(d)) → Restricted(d)].

• 3. ∃x∀d [Audited(d) → Handled(x,d)].

• 4. ContainsPatientData(R1) ∧ Audited(R1).

CNF and Skolemisation

Statement 1: ¬Employee(x) ∨ ¬Handled(x,d) ∨ ¬Restricted(d) ∨ Authorised(x).

Statement 2: ¬ContainsPatientData(d) ∨ Restricted(d).

Statement 3: ∃x∀d(¬Audited(d) ∨ Handled(x,d)). Skolemise the existential employee with a fresh constant e: ∀d(¬Audited(d) ∨ Handled(e,d)). The constant is sufficient because the existential x is outside the universal d; it does not depend on d.

Statement 4 gives unit clauses ContainsPatientData(R1) and Audited(R1). After dropping universal quantifiers, the clause set is:

• C1: ¬Employee(x) ∨ ¬Handled(x,d) ∨ ¬Restricted(d) ∨ Authorised(x)

• C2: ¬ContainsPatientData(d) ∨ Restricted(d)

• C3: ¬Audited(d) ∨ Handled(e,d)

• C4: ContainsPatientData(R1)

• C5: Audited(R1)

• C6: Employee(e) (needed to apply the stated employee rule; see note below)

Important logical observation: The four English statements as supplied do not state that the existential person in statement 3 is an employee. Therefore, they do not logically entail that an authorised employee exists. To prove the requested conclusion, the KB needs the additional fact Employee(e), or statement 3 must say “Some employee handled every document that was audited.” The assignment's statement 3 already says exactly that in English, so its correct translation includes Employee(e) as part of the existential scope: ∃x[Employee(x) ∧ ∀d(Audited(d) → Handled(x,d))]. Its Skolem clauses are Employee(e) and ¬Audited(d) ∨ Handled(e,d).

Resolution proof

From C4 and C2 with d=R1, resolve to obtain Restricted(R1). From C5 and C3 with d=R1, resolve to obtain Handled(e,R1). From Employee(e), Handled(e,R1), Restricted(R1), and C1 with x=e,d=R1, resolve successively to obtain Authorised(e).

For refutation, add ¬Authorised(e) to the clause set. Resolve it with the derived Authorised(e) to obtain the empty clause ■. Thus an authorised employee exists.

Strategy and evaluation

Set-of-support is suitable: place the negated query and clauses derived from it in the support set, and resolve support clauses against the remaining KB. Unit preference prioritises unit clauses such as the facts about R1, often shortening this proof. Record generated clauses, attempted resolvents, successful unifications, proof length and maximum clause size.

A second query such as Authorised(unknown_employee) is not entailed by these facts. A first-order prover may continue generating substitutions; use a step cap and report “unknown/cap reached”, not “false”. First-order entailment is semi-decidable: a proof can be found when one exists, but failure to find a proof need not terminate.

Skolemisation preserves satisfiability (equisatisfiability) by replacing existential witnesses with fresh Skolem terms, but it does not preserve logical equivalence in the original vocabulary. The new symbol denotes a witness and may expand the signature.

Compared with propositional resolution, first-order resolution must perform unification and manage substitutions and variable renaming. It is more expressive but generally incurs additional search overhead.

TASK 6 – FORWARD AND BACKWARD CHAINING

Organisation hierarchy KB

Use definite clauses such as ReportsTo(x,y), Manager(y), Approves(y,x), and CanApprove(y,x). Example rules: Manager(y) ← ReportsTo(x,y); SeniorManager(z) ← ReportsTo(y,z) ∧ Manager(y); CanApprove(y,x) ← ReportsTo(x,y) ∧ Manager(y); CanApprove(z,x) ← ReportsTo(x,y) ∧ ReportsTo(y,z) ∧ SeniorManager(z); Approved(x) ← CanApprove(y,x) ∧ Approves(y,x). Add about ten rules for department heads, delegated approvals and escalation.

Forward chaining

```
def forward_chain(facts, rules):
    known = set(facts)
    firings = 0
    changed = True
    while changed:
        changed = False
        for premises, conclusion in rules:
            if all(p in known for p in premises) and conclusion not in known:
                known.add(conclusion)
                firings += 1
                changed = True
    return known, firings
```

Forward chaining repeatedly applies every rule whose premises are known until no new facts are added (fixed point). It is sound for definite clauses. With a finite ground KB, it terminates because each possible fact can be added at most once.

Backward chaining

```
def backward_chain(goal, facts, rules, path=None, depth=50):
    if goal in facts: return True
    if depth <= 0: return False
    path = set() if path is None else set(path)
    if goal in path: return False       # loop check
    path.add(goal)
    for premises, conclusion in rules:
        if conclusion == goal:
            if all(backward_chain(p, facts, rules, path, depth-1)
                   for p in premises):
                return True
    return False
```

This simplified ground version demonstrates the search direction. A full FOL engine must carry substitutions through subgoals, standardise rules apart and unify each goal with rule heads.

Left recursion and remedy

A rule such as Ancestor(x,y) ← Ancestor(x,z) ∧ ReportsTo(z,y), placed before the base rule Ancestor(x,y) ← ReportsTo(x,y), can repeatedly call itself on the same goal in a naive depth-first engine. A loop check detects a repeated goal on the current path and stops that branch. Tabling/memoisation is stronger because it stores completed and in-progress subgoals.

Metrics and interpretation

For 100, 1,000 and 10,000 facts, record forward facts inferred, rule firings, redundant derivations, iterations and percentage irrelevant to the target query. For backward chaining, record goals expanded, unifications, maximum recursion depth and repeated subgoals. Forward chaining may process a large portion of the KB even if the query concerns one employee. Backward chaining starts from the requested goal and can ignore unrelated facts.

Inference direction is central: forward chaining is data-driven and is useful when many conclusions are needed or new facts arrive continuously; backward chaining is goal-driven and is useful for a small number of specific queries. Backward chaining remains efficient with many irrelevant facts if indexes restrict candidate rules and facts to those matching the current goal.

TASK 7 – HOSPITAL MEDICATION-SAFETY ADVISOR

Stage 1 – Independent proposals

Student A proposal: Use FOL because patients, drugs, allergies, conditions and interactions are naturally represented as objects and relations. Use backward chaining for a specific prescribed-drug safety query. Explain decisions by returning the rules and facts used in the proof.

Student B proposal: Use FOL with indexed predicates and forward chaining for urgent high-risk alerts, followed by backward chaining to justify a particular prescription. Store every alert with its supporting evidence and rule identifier.

Stage 2 – Comparison

Both proposals use FOL and rule-based explanations. Student A favours query-directed inference; Student B adds a forward pass for proactive alerts. The final design combines a forward-chaining alert index with backward chaining for individual prescription checks.

Stage 3 – Joint final design

Representation: FOL definite clauses plus explicit exception/override predicates. **Vocabulary:** Patient(p), Drug(d), Prescribed(p,d), HasAllergy(p,a), Contains(d,a), Takes(p,m), Interacts(d,m), HasCondition(p,c), Contraindicated(d,c), Age(p,n), MinimumAge(d,n), Dose(p,d,q), MaxDose(d,q), Safe(p,d), Unsafe(p,d), Alert(p,d,r).

Inference: Backward chaining for the query Safe(patient,drug), supported by forward-chained indexes for known allergy, interaction and contraindication alerts. **Agenda:** prioritise severe allergy and contraindication rules, then drug interactions, dose limits and age restrictions. Index facts by patient, drug and predicate.

Pseudocode

```
CHECK(patient, drug):
    evidence = []
    if allergy_match(patient, drug):
        return UNSAFE("allergy", supporting_facts)
    if contraindication_match(patient, drug):
        return UNSAFE("condition contraindication", supporting_facts)
    if interaction_match(patient, drug):
        return UNSAFE("drug interaction", supporting_facts)
    if dose_exceeds_limit(patient, drug):
        return UNSAFE("dose limit", supporting_facts)
    if age_below_minimum(patient, drug):
        return UNSAFE("age restriction", supporting_facts)
    if required_patient_data_missing(patient, drug):
        return REVIEW_REQUIRED("incomplete record")
    return SAFE("no configured rule detected a hazard")

BUILD_ALERT_INDEX(KB):
    agenda = priority_queue()
    insert newly added patient and medicine facts
    while agenda is not empty:
        fact = agenda.pop_highest_priority()
        apply matching indexed definite-clause rules
        store each new conclusion once with its rule and premises
```

The output “safe” means only that no encoded rule detected a hazard; it is not a guarantee of clinical safety. Missing data must produce review-required rather than safe.

Stage 4 – Theoretical validation

For definite clauses, forward chaining is sound and complete for deriving ground atomic consequences over a finite Herbrand base. A naive implementation can repeatedly scan all rules, costing roughly O(R×F×I), where R is rule count, F is facts and I is iterations; indexing and semi-naive evaluation reduce repeated work. Backward chaining is sound for successful proofs but naive depth-first search may loop or revisit subgoals. With tabling and suitable finite-domain assumptions it avoids repeated work.

Restricting the KB to definite clauses simplifies inference and supports efficient Horn-clause procedures, but cannot directly represent every form of negation, disjunction or uncertainty. For medical decisions, rule completeness, provenance and careful handling of missing information are essential.

Stage 4 – Experimental validation plan

Create at least 30 test cases: allergy match, no allergy, cross-reactive allergy, interacting medicine, duplicate active ingredient, contraindicated condition, pregnancy-related rule if included in the approved KB, age below/above threshold, excessive dose, missing age, missing medication list, conflicting records, emergency override, and combinations of two or more hazards. Use expected outcomes reviewed against the encoded rules.

Run at 40, 400 and 2,000 rules. For each case record query latency, inference steps, peak memory and explanation length. Compare indexed backward chaining against a baseline that scans all rules. A suitable table is:

Metric table template: Rule count | Test cases | Mean latency (ms) | Median latency (ms) | Inference steps | Peak memory (MB) | Mean explanation rules. Populate with measured values from the Python/Colab run; do not invent timings.

Stage 5 – Cross-defence

Student A explains that the experiment should show fewer irrelevant rule evaluations with goal-directed search and indexing. Student B explains that the complexity depends on the number of candidate rules/facts reached by a query, not merely the total KB size; worst-case logical inference can still be expensive.

Stage 6 – Joint evaluation

The problem suits a logical agent because many medication checks are explicit, repeatable rules and the system must provide traceable reasons. Limits include uncertain evidence, incomplete records, changing medical knowledge, dosage context and exceptions. Before choosing a knowledge-based approach, verify rule authority, coverage, consistency, update process, data quality, explainability, safety fallback, inference cost and clinical oversight.

CONTRIBUTION RECORD

The assignment requires the pair to record actual contributions. Since only one student's name was supplied, the table below is a fill-in template; enter the partner's name and the real division of work before submission.

Task	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Task 1	Knowledge-base design, model checker and definitions	Partner: validation and soundness tables	Joint review
Task 2	CNF conversion and resolution proof	Partner: model-checking comparison	Joint policy design
Task 3	FOL vocabulary and axioms	Partner: query suite and measurements	Joint debugging
Task 4	Unifier implementation and tests	Partner: MGU derivations and grounding analysis	Joint comparison
Task 5	FOL/CNF conversion and proof tree	Partner: strategy metrics and second query	Joint verification
Task 6	Forward chaining engine and metrics	Partner: backward chaining and loop remedy	Joint evaluation
Task 7	Theoretical validation	Partner: experimental validation	Joint final design

Final submission checklist

• Add the partner's name and register number if this is pair work.

• Run every Python implementation in Jupyter/Colab and attach source code, inputs and actual outputs.

• Collect performance metrics for at least three input sizes or seeds; include operation counts as primary evidence and time as secondary evidence.

• Replace the sample/analytical discussion with actual measured values and graphs.

• Ensure the contribution record reflects the work genuinely completed.